

Table of Contents

1 Kingdom Manager v3.1 LTS - Operations Manual

Release: 3.1.0 LTS

Purpose: Self-hosted Docker security intelligence, lifecycle control, staged updates, incident orchestration, recovery, rollback, configuration drift detection, validation, and reporting.

Kingdom Manager is designed to automate observation, evidence collection, vulnerability checks, low-risk playbook steps, update staging, and notifications. Destructive actions remain policy/approval gated by default.

1.1 1. What Kingdom Manager does

Kingdom Manager sits above the Docker host and coordinates four security engines:

- **Falco** - runtime behavior and syscall anomalies.
- **Trivy** - image vulnerability scanning.
- **ClamAV** - malware scanning and clamd health.
- **CrowdSec** - host intrusion decisions and firewall context.

It combines those signals with container state, container policy, known-good behavior, vulnerability results, exposure, and monitoring health. It then provides:

- a five-dimension Kingdom Security Score;
 - per-container risk scoring;
 - incident investigation and evidence capture;
 - scoped known-good approvals;
 - safe response playbooks;
 - controlled isolation and rebuild recovery;
 - staged image updates;
 - immutable image/config rollback;
 - optional Portainer Compose snapshots;
 - backup-awareness for stateful updates;
 - configuration drift detection;
 - dependency mapping;
 - a disaster recovery center;
 - Discord and n8n notifications;
 - daily/weekly reports;
 - system validation and simulation.
-

1.2 2. Safety model

Kingdom Manager intentionally separates **safe automation** from **destructive automation**.

1.2.1 Safe automation enabled by default

- container monitoring;
- Falco ingestion;
- Trivy scheduling;
- ClamAV/CrowdSec/Falco health checks;
- score recalculation;
- incident creation;
- evidence collection;
- safe incident playbook steps;
- configuration drift scans;
- update detection for containers that have Auto-Update enabled;
- candidate Trivy verification;
- Discord alerts and reports.

1.2.2 Destructive automation disabled by default

- automatic incident isolation;
- automatic incident rebuild/recovery;
- stateful automatic updates;
- automatic update application globally.

These defaults are controlled by:

`KM_PLAYBOOK_AUTO_ISOLATE: "false"`

`KM_PLAYBOOK_AUTO_RECOVER: "false"`

`KM_UPDATE_AUTO_APPLY: "false"`

`KM_UPDATE_ALLOW_STATEFUL: "false"`

Do not enable these globally until the validation and rollback tests in this manual pass on your real server.

1.3 3. First deployment or upgrade

1.3.1 Preserve existing volumes

Never delete these during an upgrade:

`kingdom-manager-data`

`kingdom-manager-trivy-cache`

The first contains Kingdom Manager's database, incidents, policies, scores, snapshots, audit log, update plans, suppressions, baselines, and validation history.

1.3.2 Recommended pre-upgrade backup

Before replacing a working release:

```
docker exec kingdom-manager sh -c \  
'cp /data/kingdom.db /data/kingdom.db.manual-backup-$(date +%s)'
```

You can also back up the Docker volume using your normal host backup system.

1.3.3 Deploy the new stack

The included compose.yaml uses:

`image: kingdom-manager:3.1.0`

Keep your existing secrets and keys. In particular, do not replace your real CrowdSec bouncer key with a blank placeholder.

After deployment:

```
docker ps --filter name=kingdom-manager \  
--format 'table {{.Names}}\t{{.Status}}\t{{.Image}}'
```

Then verify readiness:

```
curl -s http://127.0.0.1:8080/ready
```

And score integrity:

```
TOKEN="$(docker exec kingdom-manager printenv KM_API_TOKEN)"
```

```
curl -s \  
-H "Authorization: Bearer $TOKEN" \  
http://127.0.0.1:8080/api/security/score/validate
```

Finally use **Incident Center** -> **Validate** in the web UI.

1.4 4. Discord notifications

Kingdom Manager uses a Discord **webhook** for notification-only access. Discord does not receive control over your server.

1.4.1 Create a Discord webhook

In the Discord server/channel that should receive Kingdom alerts:

1. Open the channel settings.
2. Open **Integrations**.
3. Create or select a **Webhook**.

4. Copy its webhook URL.
5. Put that URL in Kingdom Manager's environment configuration.

`DISCORD_WEBHOOK_URL: "YOUR_WEBHOOK_URL"`

`KM_DASHBOARD_URL: "https://kingdom-manager-tail.kingdom.local"`

Do not post the webhook URL publicly. Anyone with the URL may be able to post into that Discord channel.

1.4.2 Optional role mention for important alerts

If you want high/critical alerts to mention a Discord role:

`DISCORD_MENTION_ROLE_ID: "123456789012345678"`

Leave it blank if you do not want mentions.

1.4.3 Notification categories

`DISCORD_NOTIFY_UPDATES: "true"`

`DISCORD_NOTIFY_RECOVERY: "true"`

`DISCORD_NOTIFY_SENSOR_FAILURES: "true"`

`KM_NOTIFY_MIN_SEVERITY: "high"`

Kingdom sends notifications for events such as:

- high/critical correlated security risk;
- container isolation;
- sensor failure and recovery;
- critical Trivy findings;
- update availability;
- completed updates;
- rollback completion;
- automatic container restart;
- daily report;
- weekly report.

1.4.4 Test Discord

After configuring the webhook, click:

Reporting & Notifications -> Test Discord

Or call:

```
curl -X POST \
  -H "Authorization: Bearer $TOKEN" \
  http://127.0.0.1:8080/api/discord/test
```

You should receive a Kingdom Manager test embed in Discord.

1.5 5. Understanding the Kingdom Security Score

The overall score is a weighted combination of five independent dimensions:

Dimension	Weight	Meaning
Threat	35%	Current effective security risk after correlation, known-good handling, and baseline context
Vulnerability	20%	Successful Trivy findings, weighted toward critical/high vulnerabilities
Exposure	15%	Published Docker host ports and higher-risk exposure of core/database/security services
Monitoring	20%	Health of Falco, Trivy, ClamAV, and CrowdSec
Trust	10%	How much active risk has evidence/context instead of being unexplained noise

The formula is intentionally designed so one failed sensor does not get counted twice and thousands of duplicate Falco alerts do not automatically become thousands of independent compromises.

1.5.1 Score interpretation

- **90-100:** Excellent
- **75-89:** Good
- **55-74:** Elevated
- **35-54:** High Risk
- **0-34:** Critical

A score is a prioritization tool, not proof that a host is compromised or safe.

1.5.2 Validate score integrity

Use **Validate** in the UI or:

```
curl -s \
-H "Authorization: Bearer $TOKEN" \
http://127.0.0.1:8080/api/security/score/validate
```

The validator checks score ranges, dimension ranges, container score ranges, and sensor-state consistency.

1.6 6. Incident workflow

When Kingdom sees meaningful correlated evidence it creates an incident.

Typical workflow:

1. **Investigate** - see correlation, confidence, Falco rules, process samples, Trivy context, ClamAV/CrowdSec evidence, and suggested action.
2. **Run Safe Steps** - capture evidence, scan, re-correlate, and observe health.
3. **Mark Expected** - only for behavior you recognize and have verified.
4. **Isolate** - removes the container from service networks and places it on the quarantine network. This is destructive to availability and remains gated.
5. **Approved Recovery** - creates and executes a controlled rebuild plan only when policy permits.
6. **Resolve** - records an operator resolution note while preserving history.

1.6.1 Automatic safe playbooks

KM_AUTO_SAFE_PLAYBOOKS=true means new incidents may automatically execute only the safe investigation steps. Kingdom de-duplicates these runs so alert storms do not continuously rescan the same incident.

The default safe actions are:

- evidence capture;
- Trivy verification;
- re-correlation;
- sensor/health observation.

Isolation and recovery remain blocked unless separately allowed.

1.7 7. Known-good behavior and baselines

1.7.1 Adaptive baseline

Kingdom observes recurring Falco behavior by exact **container + Falco rule**.

Default stability threshold:

KM_BASELINE_DAYS: "7"

KM_BASELINE_STABLE_MIN_EVENTS: "100"

KM_BASELINE_STABLE_MIN_HOURS: "12"

Learning is advisory. It does not automatically suppress a Falco rule.

1.7.2 Mark Expected

Only use **Mark Expected** after you recognize the exact behavior.

A suppression is deliberately narrow:

container = pihole

source = falco

rule = Drop and execute new binary in container

It does not suppress that rule globally.

Use **Trust** to verify that the approval is actually matching current events and reducing only the expected risk contribution.

1.8 8. Trivy scanning

Kingdom scans running container images automatically at a low rate.

Important settings:

TRIVY_AUTO_SCAN_ENABLED: "true"

TRIVY_AUTO_SCAN_EVERY_SECONDS: "1800"

TRIVY_RESCAN_SECONDS: "604800"

A failed Trivy scan is **not** treated as a clean 0/0/0 result.

The UI separates:

- last successful scan;
- latest scan attempt;
- scan errors.

1.8.1 Candidate update verification

Before a staged update is applied, the candidate image is pulled and Trivy-verified. Critical findings can block the update automatically.

1.9 9. Auto Update Engine

Auto-update is opt-in per container.

In **Container Life & Controls**, enable **Auto-Update** only on services you want Kingdom to check automatically.

1.9.1 Update rings

- **Ring 1:** test/disposable
- **Ring 2:** normal applications
- **Ring 3:** important applications
- **Ring 4:** databases/security/core infrastructure

Core services and databases are intentionally conservative.

1.9.2 Staged update flow

Detect candidate image

- > capture immutable rollback snapshot
- > preserve old image ID/configuration
- > pull candidate
- > Trivy verification
- > policy/backup checks
- > apply update
- > health observation
- > keep OR automatic rollback

1.9.3 Recommended production settings

Start with:

```
KM_UPDATE_ENGINE_ENABLED: "true"
KM_UPDATE_AUTO_APPLY: "false"
KM_UPDATE_ALLOW_STATEFUL: "false"
```

This gives automated **detection and verification**, but leaves the actual apply step under operator control.

After Ring 1 testing is successful, you may consider:

```
KM_UPDATE_AUTO_APPLY: "true"
```

Only Ring 1/2 opted-in services are automatically applied by the scheduler. Protected services remain excluded.

1.10 10. Stateful services and backups

Image rollback is not a database rollback.

If an application upgrades a database schema inside a persistent volume, restoring the old Docker image may not restore compatibility.

Therefore:

`KM_UPDATE_ALLOW_STATEFUL: "false"`

is the safe default.

If you intentionally enable stateful auto-updates, Kingdom additionally requires a recent verified backup whenever the container policy has **Backup Required** enabled.

Default maximum backup age:

`KM_BACKUP_MAX_AGE_HOURS: "48"`

Mark a backup verified through the API:

```
curl -X PUT \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"provider": "restic", "detail": "nightly backup verified"}' \
  http://127.0.0.1:8080/api/backups/CONTAINER_NAME
```

This does not create the backup itself. It tells Kingdom that your external backup system has a verified restore point.

1.11 11. Rollback and Disaster Recovery Center

Before update operations, Kingdom captures the current container's effective configuration and immutable image ID.

Captured rollback material includes:

- exact old image ID;
- image reference/tag;
- environment configuration;
- labels;
- mounts;
- restart policy;
- networks;
- port bindings;
- Docker host configuration;
- complete inspect snapshot;
- optional Compose snapshot.

1.11.1 Why immutable image IDs matter

A mutable tag such as latest can point to a different image tomorrow. Kingdom records the old sha256: image so it can restore the exact prior image.

1.11.2 Disaster Recovery Center

Open:

Incident Center -> Recovery

For every snapshot you can see:

- whether the immutable image is still present;
- whether Compose text was saved;
- whether a recent data backup is verified;
- the reason and timestamp.

Use **Test** to perform a non-destructive rollback dry-run. The test validates:

- snapshot JSON;
- old image availability;
- required Docker networks;
- reconstructable container configuration.

It does **not** stop or replace the live container.

1.11.3 Portainer Compose capture

Docker itself does not store your original Compose YAML. For exact stack-text archival, configure one of these methods.

1.11.3.1 Portainer API

PORTAINER_URL: "https://YOUR-PORTAINER"

PORTAINER_API_KEY: "YOUR_PORTAINER_API_KEY"

PORTAINER_STACK_ID: "STACK_ID"

Use a read-only/minimum-permission API key if possible.

1.11.3.2 Read-only Compose file mount

Mount your Compose file into Kingdom Manager and set:

KM_COMPOSE_SNAPSHOT_PATH: "/path/inside/container/compose.yaml"

The Docker inspect snapshot remains the rollback authority even if Portainer is unavailable.

1.12 12. Configuration Drift

Open:

Incident Center -> Drift

The first time you use Drift for a container, select **Approve Baseline** after confirming the current configuration is correct.

Kingdom then detects changes to items such as:

- privileged mode;
- published ports;
- mounts;
- Docker socket access;
- Linux capabilities;
- networks;
- security options;
- restart policy;
- read-only root filesystem;
- image reference;
- environment variable names;
- labels (stored as hashes in the drift baseline).

Dangerous changes produce a high-severity drift event and can trigger Discord notification.

1.12.1 Secrets hygiene

Drift scanning identifies environment variable **names** that look secret-bearing, such as PASSWORD, TOKEN, SECRET, or API_KEY. It does not display their values in the Drift UI.

Note: rollback snapshots necessarily contain effective runtime configuration, including environment values required to recreate a container. Protect the kingdom-manager-data volume as sensitive administrative data.

1.13 13. Dependency Map

Open:

Incident Center -> Map

Kingdom builds relationships from live Docker information:

- shared Docker networks;
- shared volumes.

Use this before stopping, isolating, rebuilding, or updating an important service to understand what else shares its infrastructure.

The graph is advisory: Docker network co-membership does not always mean an application dependency.

1.14 14. Maintenance Mode

Per-container Maintenance Mode suppresses normal security decision actions for a container while you intentionally work on it.

Kingdom also supports **global maintenance mode** through the API:

```
curl -X PUT \
-H "Authorization: Bearer $TOKEN" \
-H 'Content-Type: application/json' \
-d '{"enabled":true,"minutes":120,"reason":"host maintenance"}' \
http://127.0.0.1:8080/api/system/maintenance
```

While global maintenance is active, lifecycle auto-restart and update scheduler actions pause. Monitoring data remains available.

Disable it:

```
curl -X PUT \
-H "Authorization: Bearer $TOKEN" \
-H 'Content-Type: application/json' \
-d '{"enabled":false}' \
http://127.0.0.1:8080/api/system/maintenance
```

1.15 15. System Validation and Simulation

Open:

Incident Center -> Validate

Validation checks include:

- SQLite database integrity;
- schema version;
- Docker API reachability;
- Falco health;
- ClamAV health;
- CrowdSec health;
- Trivy health;
- score bounds;
- dimension bounds;
- core self-protection;
- destructive automation defaults;
- stateful-update safety;
- Discord configuration status.

The Discord configuration check is informational and does not make core validation fail.

1.15.1 Simulation

The Validation drawer includes non-destructive simulations for:

- Falco-only incident;
- multi-source incident;
- update/rollback failure.

Simulation predicts Kingdom's policy path. It does not inject real malware, stop containers, or change production state.

Additional API scenarios:

falco-only
multi-source
malware
sensor-outage
update-failure

1.16 16. Sensor troubleshooting

1.16.1 ClamAV

Kingdom checks clamd using the protocol-level command:

```
zPING\0 -> PONG\0
```

If ClamAV shows down even though its container is running, test from Kingdom Manager:

```
docker exec -i kingdom-manager python - <<'PY'
import socket
with socket.create_connection(("clamav",3310),5) as s:
    s.sendall(b"zPING\0")
    print(repr(s.recv(64)))
PY
```

Expected:

```
b'PONG\x00'
```

1.16.2 CrowdSec

Make sure the bouncer API key is configured:

```
CROWDSEC_URL: "http://172.24.0.1:8080"
CROWDSEC_API_KEY: "YOUR_EXISTING_KINGDOM_MANAGER_BOUNCER_KEY"
```

Do not share that key publicly.

1.16.3 Falco

Confirm both containers share the security network and that Falco's health port is reachable:

```
docker exec -i kingdom-manager python - <<'PY'
import socket
s=socket.create_connection(("falco",8765),5)
print("Falco health connection: OK")
s.close()
PY
```

1.16.4 Trivy

```
docker exec kingdom-manager-trivy trivy --version
```

A failed scan remains an error until a later successful result is recorded.

1.17 17. Policies and recommended profiles

1.17.1 Test / disposable

- Auto-Restart: ON
- Auto-Update: ON after rollback testing
- Auto-Isolate: optional
- Approved Rebuild: optional
- Protected: OFF
- Ring: 1

1.17.2 Normal user application

- Auto-Restart: ON
- Auto-Update: ON after confidence is established
- Auto-Isolate: OFF initially
- Approved Rebuild: OFF until recovery is tested
- Protected: OFF
- Ring: 2

1.17.3 Important application

- Auto-Restart: ON
- Auto-Update: staged/manual apply
- Auto-Isolate: OFF
- Approved Rebuild: carefully tested
- Protected: optional
- Ring: 3

1.17.4 Database

- Protected: recommended
- Auto-Update: OFF
- Automatic recovery: OFF by default
- Backup Required: ON
- Ring: 4

1.17.5 Kingdom/security infrastructure

Kingdom Manager, Docker API proxies, Falco, ClamAV, CrowdSec, Trivy, Portainer, and reverse-proxy infrastructure should normally remain protected and Ring 4.

1.18 18. Suggested path to higher automation

Do not enable everything on day one.

1.18.1 Stage A - Observe

Leave:

```
KM_UPDATE_AUTO_APPLY: "false"
KM_PLAYBOOK_AUTO_ISOLATE: "false"
KM_PLAYBOOK_AUTO_RECOVER: "false"
KM_UPDATE_ALLOW_STATEFUL: "false"
```

Run for several days and tune known-good Falco behavior.

1.18.2 Stage B - Ring 1 updates

Enable Auto-Update on one disposable container. Verify:

1. update detection;
2. snapshot creation;
3. candidate scan;
4. manual apply;
5. health observation;
6. manual rollback;
7. Disaster Recovery **Test** passes.

1.18.3 Stage C - Ring 2 auto apply

After Ring 1 succeeds repeatedly, optionally set:

```
KM_UPDATE_AUTO_APPLY: "true"
```

Only explicitly opted-in, non-protected Ring 1/2 services are eligible.

1.18.4 Stage D - Isolation/recovery

Only after incident playbooks and backups are proven should you consider enabling automated isolation/recovery on selected disposable applications.

Do not globally enable automated database recovery unless you have application-aware backups and restore testing.

1.19 19. Recommended daily workflow

1. Check **Kingdom Security Score** and five posture dimensions.
 2. Review **Immediate Attention**.
 3. Open new incidents and review safe playbook results.
 4. Review **Recommendations**.
 5. Check **Drift** for dangerous configuration changes.
 6. Check **Updates** for verified candidates.
 7. Check **Recovery** before applying important updates.
 8. Let Discord notify you about high-priority changes instead of continuously watching the dashboard.
-

1.20 20. Recommended weekly workflow

1. Read the weekly Discord report.
 2. Review unresolved incidents.
 3. Review Trust approvals nearing expiration.
 4. Review failed Trivy scans.
 5. Run **Validate**.
 6. Test at least one recent rollback snapshot for important services.
 7. Confirm database/application backups are restorable outside Kingdom Manager.
 8. Review new published ports and Drift warnings.
-

1.21 21. Emergency rules

If you believe the server is actively compromised:

1. Do not blindly mark alerts Expected.
 2. Capture incident evidence first.
 3. Use isolation only on the affected workload unless you intentionally want an outage.
 4. Preserve snapshots and logs.
 5. Do not auto-update a compromised service as a substitute for investigation.
 6. Treat leaked API tokens/webhooks as compromised credentials and rotate them.
 7. For stateful services, verify data backups before rebuild/rollback.
-

1.22 22. Important limitations

Kingdom Manager cannot guarantee that a container image rollback will reverse changes made to persistent application data.

Dependency mapping is inferred from Docker topology, not application-level protocol tracing.

Configuration drift baselines tell you that configuration changed; they do not prove whether the change is malicious.

Trivy vulnerability results describe known vulnerability metadata, not whether a vulnerability was exploited.

Falco severity is sensor severity. Kingdom correlation is the layer that determines effective server risk.

A green dashboard is not a replacement for offline backups, host patching, account security, or network security.

1.23 23. Core API quick reference

GET /health

GET /ready

GET /api/security/score

GET /api/security/score/validate

GET /api/security/posture

GET /api/system/validate

GET /api/dependencies

GET /api/drift

POST /api/drift/{container}/approve

GET /api/disaster-recovery

POST /api/disaster-recovery/{snapshot_id}/test

GET /api/updates

POST /api/updates/{container}/check

POST /api/updates/{plan_id}/verify

POST /api/updates/{plan_id}/apply

POST /api/updates/{plan_id}/rollback

GET /api/backups

PUT /api/backups/{container}

POST /api/discord/test

POST /api/simulate/{scenario}

PUT /api/system/maintenance

All /api/ endpoints require the Kingdom API token unless explicitly documented otherwise.

1.24 24. Production-ready checklist

Before calling your installation fully automated, confirm all of these:

- ☐ `/ready` returns OK.
- ☐ **Validate** passes all core checks.
- ☐ Falco is healthy and receiving events.
- ☐ ClamAV returns PONG through Kingdom Manager.
- ☐ CrowdSec API authentication succeeds.
- ☐ Trivy completes successful scans.
- ☐ Discord test arrives.
- ☐ One disposable update has been staged and applied.
- ☐ That disposable update has been rolled back successfully.
- ☐ Disaster Recovery dry-run passes for the test snapshot.
- ☐ Core/security/database services are protected appropriately.
- ☐ Configuration baselines are approved for stable important containers.
- ☐ Dangerous Drift alerts reach Discord.
- ☐ Stateful services have real external backups.
- ☐ Stateful auto-update remains disabled unless restores have been tested.
- ☐ You understand which services may be automatically restarted, updated, isolated, or rebuilt.

When those checks pass, Kingdom Manager can safely automate the repetitive work while keeping high-impact actions inside explicit policy boundaries.